**Day 23–24: Arrays**

- One-dimensional arrays
- Two-dimensional arrays
- Array operations

Problems:

- Find maximum
- Sorting
- Searching
- Matrix problems

**1. First: Array ante enti?**

Suppose neeku 5 students marks store cheyyali.

Normal variables use chesthe:

int mark1 = 80;

int mark2 = 75;

int mark3 = 90;

int mark4 = 65;

int mark5 = 88;

Idi possible, but **5 values ki 5 variables** create cheyyali.

Suppose 1000 marks unte?

int mark1;

int mark2;

int mark3;

...

int mark1000;

Chaala difficult.

So Java lo **Array** use chestham.

**Array definition**

**Array is a collection of multiple values of the same data type stored under one variable name.**

Example:

int[] marks = {80, 75, 90, 65, 88};

Ikkada:

- marks → array variable
- int → array lo store ayye data type
- 80, 75, 90, 65, 88 → elements
- 5 → array size

So one variable marks lo 5 integers store chesam.

---

**2. Real-life Example**

Imagine oka apartment building.

There are 5 rooms:

Room 0 → 80

Room 1 → 75

Room 2 → 90

Room 3 → 65

Room 4 → 88

Array lo each element ki oka **index** untundi.

**Important:**

**Array index always 0 nunchi start avtundi.**

Value: 80 75 90 65 88

Index: 0 1 2 3 4

So:

marks[0]

means first element → 80

marks[2]

means third element → 90

**Why 0?**

Java and many programming languages lo indexing convention **0-based indexing**.

Therefore:

For an array of size 5:

First index = 0

Last index = 4

General formula:

Last index = size - 1

---

**3. Array Create cheyyadaniki different ways**

**Method 1: Direct initialization**

int[] numbers = {10, 20, 30, 40, 50};

This is the easiest way.

---

**Method 2: First size declare cheyyadam**

int[] numbers = new int[5];

Ikkada size 5.

Java automatically default values assign chestundi.

numbers[0] = 0

numbers[1] = 0

numbers[2] = 0

numbers[3] = 0

numbers[4] = 0

Because int default value is 0.

Then values assign cheyyachu:

numbers[0] = 10;

numbers[1] = 20;

numbers[2] = 30;

numbers[3] = 40;

numbers[4] = 50;

---

**4. Array declaration syntax**

int[] numbers;

Ikkada just declaration.

Array create cheyyaledu.

Create cheyyali:

numbers = new int[5];

Or directly:

int[] numbers = new int[5];

---

**5. int[] vs int numbers[]**

Both valid:

int[] numbers;

and

int numbers[];

But generally Java coding lo:

int[] numbers;

is preferred.

---

**6. Array size**

Suppose:

int[] numbers = {10, 20, 30, 40};

Array size telusukovali ante:

numbers.length

Output:

4

**Important difference**

For arrays:

numbers.length

For String:

name.length()

Array ki length **property**.

String ki length() **method**.

---

**7. Accessing Array Elements**

int[] numbers = {10, 20, 30, 40, 50};

System.out.println(numbers[0]);

System.out.println(numbers[1]);

System.out.println(numbers[2]);

Output:

10

20

30

---

**8. Updating an Array Element**

Suppose:

int[] numbers = {10, 20, 30, 40, 50};

Third value 30 ni 100 ga change cheyyali.

numbers[2] = 100;

Now:

10 20 100 40 50

Array elements **mutable**.

Meaning existing value ni change cheyyachu.

---

**9. Traversing an Array**

**Traversal** ante array lo unna every element ni one by one visit cheyyadam.

Most commonly for loop use chestham.

```
int[] numbers = {10, 20, 30, 40, 50};

for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

**Step-by-step**

First:

i = 0

Print:

numbers[0]

Then:

i = 1

Print:

numbers[1]

and so on.

---

**10. Enhanced for loop**

Java lo another easy way:

```
int[] numbers = {10, 20, 30, 40, 50};

for (int num : numbers) {
    System.out.println(num);
}
```

Meaning:

numbers lo unna each value ni num lo temporarily store chesi process cheyyi.

Output:

10

20

30

40

50

**Difference**

Normal for:

for (int i = 0; i < numbers.length; i++)

Index kavali ante useful.

Enhanced for:

for (int num : numbers)

Just values kavali ante useful.

---

**11. Taking Array Input from User**

Very important for coding tests.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] numbers = new int[n];

        for (int i = 0; i < n; i++) {
            numbers[i] = sc.nextInt();
        }
```

```
        for (int i = 0; i < n; i++) {
            System.out.println(numbers[i]);
        }
    }
}
```

Suppose input:

5

10

20

30

40

50

Then:

n = 5

Array:

[10, 20, 30, 40, 50]

---

**12. Important Array Rule**

Suppose:

int[] arr = new int[5];

Valid indexes:

0

1

2

3

4

Invalid:

arr[5]

Because index 5 doesn't exist.

It causes:

ArrayIndexOutOfBoundsException

This is one of the most common beginner mistakes.

---

**□ ONE-DIMENSIONAL ARRAY**

**13. What is a 1D Array?**

One-dimensional array means **single sequence/list of elements**.

Example:

int[] arr = {10, 20, 30, 40, 50};

Visual:

Index → 0 1 2 3 4

↓ ↓ ↓ ↓ ↓

Array → 10 20 30 40 50

Think of it as **one row**.

---

**14. Basic Operations on 1D Array**

Common operations:

1. Traversal
2. Access
3. Update
4. Insertion
5. Deletion
6. Searching
7. Sorting
8. Finding maximum/minimum
9. Finding sum
10. Reversing

Let's understand each.

---

**15. Traversal**

```
int[] arr = {5, 10, 15, 20};

for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

---

**16. Find Sum**

```
int[] arr = {10, 20, 30, 40};

int sum = 0;

for (int i = 0; i < arr.length; i++) {
    sum = sum + arr[i];
}

System.out.println(sum);
```

Output:

100

**Logic**

Initially:

sum = 0

Then:

sum = 0 + 10 = 10

sum = 10 + 20 = 30

sum = 30 + 30 = 60

sum = 60 + 40 = 100

---

**□ PROBLEM 1: FIND MAXIMUM**

This is a **very important interview problem**.

Suppose:

int[] arr = {10, 50, 30, 90, 20};

Maximum is:

90

---

**Wrong approach beginners often use**

Some people do:

int max = 0;

This can create problems if all numbers are negative.

Example:

-10 -20 -5 -30

Maximum is:

-5

But if:

max = 0

you might incorrectly get 0.

---

**Correct approach**

Take the first element as maximum:

int max = arr[0];

Then compare remaining elements.

int[] arr = {10, 50, 30, 90, 20};

int max = arr[0];

for (int i = 1; i < arr.length; i++) {

    if (arr[i] > max) {

```
        max = arr[i];
    }
}

System.out.println(max);
```

Output:

90

---

**Understand the logic deeply**

Initially:

max = 10

Compare 50:

50 > 10

Yes.

So:

max = 50

Compare 30:

30 > 50

No.

So:

max = 50

Compare 90:

90 > 50

Yes.

max = 90

Compare 20:

20 > 90

No.

Final:

max = 90

**General pattern**

Remember this pattern:

```
int max = arr[0];

for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
```

You will use this pattern many times.

---

**□ Find Minimum**

Same concept.

```
int[] arr = {10, 50, 30, 90, 20};

int min = arr[0];

for (int i = 1; i < arr.length; i++) {

    if (arr[i] < min) {
        min = arr[i];
    }
}

System.out.println(min);
```

Output:

10

---

**□ SEARCHING**

Searching ante:

Array lo oka particular value unda? Unte ekkada undi?

Example:

10 20 30 40 50

Search:

30

Answer:

Found at index 2

---

**17. Linear Search**

Array sorted undalsina requirement **ledu**.

Example:

```
int[] arr = {10, 40, 20, 50, 30};

int target = 50;
```

One by one check chestham.

```
int index = -1;

for (int i = 0; i < arr.length; i++) {

    if (arr[i] == target) {
        index = i;
        break;
    }
}

System.out.println(index);
```

Output:

3

---

**Why index = -1?**

Because valid array indexes are:

0, 1, 2, 3...

So -1 can represent:

Element not found.

Example:

10 20 30

Search:

50

No match.

Therefore:

index = -1

---

**18. Search and return Found/Not Found**

```
boolean found = false;

for (int i = 0; i < arr.length; i++) {

    if (arr[i] == target) {
        found = true;
        break;
    }
}

if (found) {
    System.out.println("Found");
} else {
```

```
    System.out.println("Not Found");
}
```

---

**□ SORTING**

Sorting means arranging elements in a particular order.

**Ascending**

Small → Big

10 20 30 40 50

**Descending**

Big → Small

50 40 30 20 10

---

**19. Bubble Sort**

For beginners, Bubble Sort is very useful to understand sorting logic.

Suppose:

5 3 8 1 2

We compare neighboring elements.

If left > right, swap.

---

**First pass**

5 3 8 1 2

Compare:

5 and 3

5 > 3 → swap

3 5 8 1 2

Compare:

5 and 8

No swap.

3 5 8 1 2

Compare:

8 and 1

Swap:

3 5 1 8 2

Compare:

8 and 2

Swap:

3 5 1 2 8

Largest element 8 reached the end.

This is why it's called **Bubble Sort** — larger elements gradually "bubble" toward the end.

---

**20. Bubble Sort Java Code**

```
int[] arr = {5, 3, 8, 1, 2};


for (int i = 0; i < arr.length - 1; i++) {


    for (int j = 0; j < arr.length - 1 - i; j++) {


        if (arr[j] > arr[j + 1]) {


            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

Print:

```
for (int num : arr) {
```

```
    System.out.print(num + " ");
}
```

Output:

1 2 3 5 8

---

**21. Why temp?**

Suppose:

a = 5

b = 3

We want:

a = 3

b = 5

If directly:

a = b;

b = a;

After first line:

a = 3

b = 3

Original 5 lost.

So temporary variable:

int temp = a;

a = b;

b = temp;

Think:

temp = 5

a = 3

b = 5

---

**22. Java Built-in Sorting**

Real projects lo manually Bubble Sort raayalsina requirement usually undadu.

Java provides:

Arrays.sort(arr);

Need import:

import java.util.Arrays;

Example:

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {

        int[] arr = {5, 3, 8, 1, 2};

        Arrays.sort(arr);

        System.out.println(Arrays.toString(arr));
    }
}
```

Output:

[1, 2, 3, 5, 8]

**But interview preparation lo:**

**Bubble Sort logic definitely understand cheyyali.**

Because sorting algorithms concepts are important.

---

**□ TWO-DIMENSIONAL ARRAY**

Now 2D arrays.

**23. What is 2D Array?**

1D:

10 20 30 40

2D:

10 20 30

40 50 60

70 80 90

It has:

- rows
- columns

Think of it as a **table/matrix**.

---

**24. Real-Life Example**

Student marks table:

| | Subject1 | Subject2 | Subject3 |
|---|---|---|---|
| Student1 | 80 | 90 | 70 |
| Student2 | 75 | 85 | 95 |
| Student3 | 88 | 92 | 81 |

This is 2D data.

---

**25. Creating a 2D Array**

```
int[][] matrix = new int[3][3];
```

Means:

3 rows

3 columns

Visual:

```
      Col
     0  1  2

Row 0  _  _  _
Row 1  _  _  _
Row 2  _  _  _
```

---

**26. Direct initialization**

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Visual:

1 2 3

4 5 6

7 8 9

---

**27. Accessing 2D Array**

Syntax:

matrix[row][column]

Example:

matrix[0][0]

→ 1

matrix[1][2]

→ 6

matrix[2][1]

→ 8

**Remember:**

First index = **row**

Second index = **column**

matrix[row][column]

---

**28. 2D Array Traversal**

2D array ki usually **nested loops** use chestham.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < matrix.length; i++) {

    for (int j = 0; j < matrix[i].length; j++) {

        System.out.print(matrix[i][j] + " ");
    }

    System.out.println();
}
```

Output:

1 2 3

4 5 6

7 8 9

---

**29. Understand i and j**

This is extremely important.

Usually:

i = row

j = column

For:

1 2 3

4 5 6

7 8 9

First:

i = 0

We are in first row:

1 2 3

Then j:

j = 0 → 1

j = 1 → 2

j = 2 → 3

Then:

i = 1

Second row:

4 5 6

and so on.

---

**□ MATRIX PROBLEMS**

A matrix is basically a rectangular arrangement of numbers in rows and columns.

Many coding interview questions use matrices.

---

**30. Matrix Addition**

Suppose:

A = B =

1 2 5 6

3 4 7 8

Add corresponding positions:

1+5 = 6

2+6 = 8

3+7 = 10

4+8 = 12

Result:

6  8

10 12

Java:

```
int[][] A = {
    {1, 2},
    {3, 4}
};

int[][] B = {
    {5, 6},
    {7, 8}
};

int[][] result = new int[2][2];

for (int i = 0; i < 2; i++) {

    for (int j = 0; j < 2; j++) {

        result[i][j] = A[i][j] + B[i][j];
    }
}
```

---

**31. Matrix Sum**

Suppose:

1 2 3

4 5 6

7 8 9

Total:

45

Code:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

int sum = 0;

for (int i = 0; i < matrix.length; i++) {

    for (int j = 0; j < matrix[i].length; j++) {

        sum += matrix[i][j];
    }
}

System.out.println(sum);
```

---

**32. Row Sum**

Matrix:

1 2 3

4 5 6

7 8 9

Row 0:

1 + 2 + 3 = 6

Row 1:

4 + 5 + 6 = 15

Row 2:

7 + 8 + 9 = 24

Code:

```
for (int i = 0; i < matrix.length; i++) {

    int sum = 0;

    for (int j = 0; j < matrix[i].length; j++) {
        sum += matrix[i][j];
    }

    System.out.println(sum);
}
```

Output:

6

15

24

---

**33. Main Diagonal**

For:

1 2 3

4 5 6

7 8 9

Main diagonal:

1

  5

    9

Positions:

[0][0]

[1][1]

[2][2]

Notice:

row == column

So:

```
for (int i = 0; i < matrix.length; i++) {
    System.out.println(matrix[i][i]);
}
```

---

**34. Main Diagonal Sum**

```
int sum = 0;

for (int i = 0; i < matrix.length; i++) {
    sum += matrix[i][i];
}

System.out.println(sum);
```

For:

1 2 3

4 5 6

7 8 9

Result:

1 + 5 + 9 = 15

---

**35. Secondary Diagonal**

Same matrix:

1 2 3

4 5 6

7 8 9

Secondary diagonal:

  3

 5

7

Positions:

[0][2]

[1][1]

[2][0]

Formula:

column = n - 1 - row

Code:

```
int n = matrix.length;


for (int i = 0; i < n; i++) {
    System.out.println(matrix[i][n - 1 - i]);
}
```

Output:

3

5

7

---

**36. Matrix Transpose**

Transpose ante:

Rows ni columns ga, columns ni rows ga change cheyyadam.

Original:

1 2 3

4 5 6

Transpose:

1 4

2 5

3 6

Notice:

Original matrix[i][j]

becomes:

Transpose[j][i]

Code:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

int rows = matrix.length;
int cols = matrix[0].length;

int[][] transpose = new int[cols][rows];

for (int i = 0; i < rows; i++) {

    for (int j = 0; j < cols; j++) {

        transpose[j][i] = matrix[i][j];
    }
}
```

---

**37. 2D Array Input**

Suppose user gives:

2

3

1 2 3

4 5 6

Meaning:

rows = 2

columns = 3

Java:

```
Scanner sc = new Scanner(System.in);

int rows = sc.nextInt();
int cols = sc.nextInt();

int[][] matrix = new int[rows][cols];

for (int i = 0; i < rows; i++) {

    for (int j = 0; j < cols; j++) {

        matrix[i][j] = sc.nextInt();
    }
}
```

---

**□ ARRAY INSERTION**

Arrays have a fixed size.

Suppose:

```
int[] arr = new int[5];
```

Its capacity is 5.

Unlike an ArrayList, normal Java array size cannot automatically grow.

If you want to "insert" while maintaining a particular logical size, you usually shift elements manually or create a larger array.

Example:

10 20 30 40

Insert 25 at index 2.

Desired:

10 20 25 30 40

Need shifting:

40 → right

30 → right

Then:

25 → index 2

---

**□ ARRAY DELETION**

Suppose:

10 20 30 40 50

Delete 30.

Shift left:

10 20 40 50

Normal array's physical size remains the same, but we can maintain a smaller **logical size**.

This distinction is important.

---

**□ IMPORTANT: Array vs ArrayList**

You will learn ArrayList later, but remember this basic difference:

| **Array** | **ArrayList** |
|---|---|
| Fixed size | Dynamic size |
| int[] | ArrayList<Integer> |
| Primitive types allowed | Objects/wrapper types |

| Array | ArrayList |
| --- | --- |
| Fast/simple | More flexible |
| Size fixed after creation | Can grow/shrink |

Example:

int[] arr = new int[5];

Size cannot become 6 automatically.

---

**□ ARRAY OF DIFFERENT DATA TYPES**

Arrays can be created for different types.

int[] numbers;

double[] prices;

char[] letters;

boolean[] flags;

String[] names;

Example:

String[] names = {"Ram", "Sam", "John"};

---

**□ ARRAY OF OBJECTS**

Arrays don't have to contain only primitive values.

Example:

String[] names = new String[3];

Each position can contain a String reference.

Similarly, later when you learn classes:

Student[] students = new Student[5];

This is an **array of object references**.

---

**□ DEFAULT VALUES**

When you create:

int[] arr = new int[5];

Default:

0 0 0 0 0

For different types:

| Type | Default value |
|---|---|
| int | 0 |
| double | 0.0 |
| float | 0.0f |
| char | '\u0000' |
| boolean | false |
| Object/String reference | null |

---

**□ ARRAY MEMORY CONCEPT**

You don't need to become a memory expert now, but understand the basic idea.

When you write:

int[] arr = {10, 20, 30};

arr refers to an array object.

Conceptually:

arr

↓

+----+----+----+

| 10 | 20 | 30 |

+----+----+----+

0 1 2

arr[0] accesses first element.

arr[2] accesses third element.

The array object has a fixed length.

---

## □ COMMON ARRAY MISTAKES

**Mistake 1: Starting index at 1**

Wrong:

```
for (int i = 1; i <= arr.length; i++)
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Why?

For size 5:

Indexes = 0,1,2,3,4

---

**Mistake 2: Using <=**

Wrong:

```
i <= arr.length
```

Correct:

```
i < arr.length
```

Because when:

```
i = arr.length
```

you're already outside the valid index range.

---

**Mistake 3: Confusing length and length()**

Array:

```
arr.length
```

String:

```
str.length()
```

---

**Mistake 4: Wrong 2D indexes**

Remember:

```
matrix[row][column]
```

Not:

matrix[column][row]

unless your specific logic requires it.

---

**□ IMPORTANT ARRAY PATTERNS**

You should memorize **logic patterns**, not blindly memorize entire programs.

**Pattern 1 — Traversal**

```
for (int i = 0; i < arr.length; i++) {
    // arr[i]
}
```

**Pattern 2 — Maximum**

```
int max = arr[0];

for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
```

**Pattern 3 — Minimum**

```
int min = arr[0];

for (int i = 1; i < arr.length; i++) {
    if (arr[i] < min) {
        min = arr[i];
    }
}
```

**Pattern 4 — Sum**

```
int sum = 0;

for (int i = 0; i < arr.length; i++) {
```

```
    sum += arr[i];
}
```

**Pattern 5 — Search**

```
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) {
        // found
    }
}
```

**Pattern 6 — 2D traversal**

```
for (int i = 0; i < matrix.length; i++) {

    for (int j = 0; j < matrix[i].length; j++) {

        // matrix[i][j]
    }
}
```

These six patterns become **very important for DSA**.

---

**⏱ TIME COMPLEXITY — Very Important**

You should start learning this along with arrays.

**Traversal**

```
for (int i = 0; i < n; i++)
```

Time:

O(n)

Because if there are n elements, we visit n elements.

---

**Linear Search**

Worst case:

O(n)

Because target may be at the end or absent.

---

**Find Maximum**

O(n)

We check every element.

---

**Bubble Sort**

Basic Bubble Sort:

$O(n^2)$

Because there are nested loops.

For example:

$n \times n$

approximately.

---

**Matrix Traversal**

For an $r \times c$ matrix:

$O(r \times c)$

Because every cell is visited.

For a square $n \times n$ matrix:

$O(n^2)$

---

**□ PRACTICE PROBLEMS**

Once you understand the above, practice these in this order:

**Beginner**

1. Print all array elements
2. Find sum
3. Find average
4. Find maximum
5. Find minimum

6. Count even numbers
7. Count odd numbers
8. Count positive numbers
9. Count negative numbers
10. Search an element

**Intermediate**

11. Reverse an array
12. Find second largest
13. Find second smallest
14. Count frequency of an element
15. Remove duplicates
16. Check whether array is sorted
17. Find duplicate elements
18. Find missing number
19. Move zeros to the end
20. Merge two arrays

**Sorting**

21. Bubble Sort
22. Selection Sort
23. Insertion Sort
24. Arrays.sort()
25. Sort ascending
26. Sort descending

**2D / Matrix**

27. Print matrix
28. Matrix sum
29. Matrix addition
30. Row-wise sum
31. Column-wise sum

32. Main diagonal sum
33. Secondary diagonal sum
34. Transpose
35. Search an element in matrix
36. Find maximum in matrix
37. Print boundary elements
38. Rotate matrix
39. Matrix multiplication

---

## 🎯 DAY 23–24 FINAL CHEAT SHEET

### 1D Array

int[] arr = {10, 20, 30};

Access:

arr[0]

Size:

arr.length

Update:

arr[1] = 100;

Traversal:

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Enhanced loop:

```
for (int x : arr) {
    System.out.println(x);
}
```

---

### Maximum

int max = arr[0];

```
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
```

---

**Minimum**

```
int min = arr[0];

for (int i = 1; i < arr.length; i++) {
    if (arr[i] < min) {
        min = arr[i];
    }
}
```

---

**Linear Search**

```
int index = -1;

for (int i = 0; i < arr.length; i++) {

    if (arr[i] == target) {
        index = i;
        break;
    }
}
```

---

**Built-in Sorting**

```
import java.util.Arrays;
```

```
Arrays.sort(arr);
```

---

**2D Array**

```
int[][] matrix = new int[3][3];
```

or:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Access:

```
matrix[row][column]
```

Traversal:

```
for (int i = 0; i < matrix.length; i++) {

    for (int j = 0; j < matrix[i].length; j++) {

        System.out.print(matrix[i][j] + " ");
    }

    System.out.println();
}
```

---

**☆ The most important things you should understand today**

Don't try to memorize 20 programs. First make these concepts crystal clear:

Array

↓

Collection of same-type elements

↓

Index starts at 0

↓

Last index = length - 1

↓

arr[index]

↓

arr.length

↓

for loop → traversal

↓

max/min → comparison

↓

search → compare with target

↓

sorting → arrange elements

↓

2D array → rows + columns

↓

matrix[row][column]

↓

nested loops → matrix traversal

If these are clear, **arrays are no longer a scary topic**. The next step in your Java/DSA preparation should be solving the problems above **without looking at the solution**, especially **maximum, minimum, linear search, reverse, second largest, duplicate, missing number, and matrix problems**.